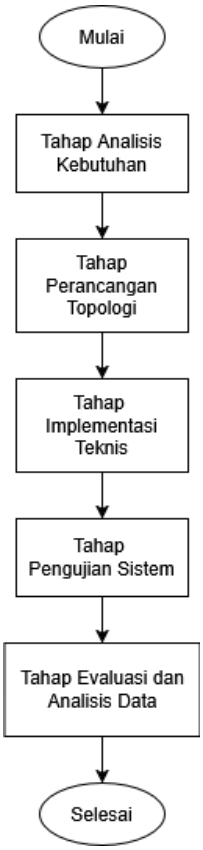
BAB III
METODOLOGI PENELITIAN

3.1 Metode Penelitian

Metode penelitian ini disusun secara sistematis untuk menguji keandalan sistem otomatisasi *backup* pada jaringan *hybrid* sistem pemeriksaan pasien rawat inap. Pendekatan yang digunakan adalah penelitian kuantitatif eksperimental yang mencakup tahapan analisis kebutuhan, perancangan topologi, hingga implementasi teknik replikasi *master-slave* pada lingkungan *Virtual Private Server* (VPS). Melalui penetapan variabel yang terukur, penelitian ini bertujuan untuk menganalisis efisiensi durasi *backup* serta stabilitas sinkronisasi data medis secara otomatis guna menjamin konsistensi dan ketersediaan informasi yang berkelanjutan.

3.1.1 Tahapan penelitian

Alur metodologi penelitian dapat dilihat pada *flowchart* gambar 3.1 berikut,



Gambar 3.1 *Flowchart* Metodologi Penelitian

Dilihat pada gambar 3.1 yang menunjukkan metodologi yang digunakan adalah adaptasi dari model pengembangan sistem yang sistematis:

a. Tahap analisis kebutuhan

Peneliti melakukan identifikasi data yang diterima dari jaringan *hybrid* sistem pemeriksaan pasien rawat inap. Rasionalnya adalah untuk menentukan volume data dan frekuensi penulisan agar jadwal *cron job* dapat diatur secara optimal guna menghindari beban kerja berlebih pada server.

b. Tahap perancangan topologi

Merancang arsitektur *master-slave* yang menugaskan server utama (*Master*) menangani transaksi data pasien secara *real-time*, sedangkan server sekunder (*Slave*) didedikasikan untuk menerima distribusi data dan proses *backup*. Hal ini memastikan sumber daya *Master* tidak terganggu oleh proses pencadangan.

c. Tahap implementasi teknis

1) Konfigurasi replikasi: Mengaktifkan *binary logging* pada server *master* dan mengonfigurasi server *Slave* untuk menyalin perubahan data secara otomatis.
2) Integrasi mariadb-*backup*: Mengimplementasikan fitur *hot backup* agar penyalinan data dilakukan tanpa mengunci tabel basis data (*non-locking*), sehingga layanan pasien tetap aktif.
3) Otomatisasi *cron job:* Mengatur penjadwalan perintah eksekusi *backup* di Linux agar berjalan secara konsisten tanpa intervensi manual.

d. Tahap pengujian sistem

Pengujian sistem diimplementasikan dengan mensimulasikan *input* data pasien pada jaringan *hybrid* menuju server *Master*. Selanjutnya, dilakukan pemantauan terhadap status replikasi untuk memverifikasi bahwa proses penyalinan data ke server *Slave* telah berlangsung secara otomatis tanpa hambatan.

e. Tahap evaluasi dan analisis data

Pengumpulan data temporal dari setiap siklus pencadangan otomatis dilakukan untuk dianalisis menggunakan metode *Average Time*. Melalui metode ini, efektivitas durasi otomatisasi dapat diukur secara kuantitatif untuk membuktikan

konsistensi sistem dalam melakukan sinkronisasi data medis di tengah fluktuasi beban kerja jaringan *hybrid*.

3.1.2 Metode pengumpulan dan analisis data

Penelitian ini menggunakan pendekatan eksperimental dengan tujuan menganalisis efektivitas otomatisasi *backup* data pada lingkungan jaringan *hybrid* sistem rawat inap rumah sakit. Metode eksperimen dipilih karena penelitian ini melibatkan perancangan, implementasi, serta pengujian langsung terhadap sistem replikasi basis data dan mekanisme pencadangan otomatis untuk memperoleh data kinerja yang terukur dan objektif.

a. Metode replikasi *master–slave*

Metode utama yang digunakan dalam penelitian ini adalah replikasi basis data MySQL dengan arsitektur *master–slave*. Pada metode ini, server *Master* berfungsi sebagai pusat transaksi yang melayani operasi baca dan tulis data, sedangkan server *Slave* berperan sebagai server cadangan yang mereplikasi setiap perubahan data dari *Master* secara otomatis. Replikasi ini diterapkan pada lingkungan jaringan *hybrid* yang mengombinasikan infrastruktur lokal dan VPS, sehingga data pemeriksaan pasien rawat inap tetap tersinkronisasi dan tersedia meskipun terjadi gangguan pada salah satu server. Implementasi replikasi bertujuan untuk meningkatkan ketersediaan data (*availability*) serta meminimalkan risiko *single point of failure* pada sistem basis data rumah sakit.

b. Metode otomatisasi *backup*

Untuk menjamin keberlangsungan data, proses pencadangan dilakukan secara otomatis menggunakan mariadb-*backup* yang dijalankan pada server *Slave.* Pemanfaatan *hot backup* memungkinkan proses pencadangan dilakukan tanpa menghentikan layanan basis data atau mengunci tabel, sehingga tidak mengganggu aktivitas input data pasien yang berlangsung secara *real-time*. Penjadwalan *backup* dilakukan menggunakan *cron job* pada sistem operasi Linux, dengan interval waktu tertentu (misalnya setiap pukul 02.00 WIB), sehingga proses *backup* berjalan secara konsisten tanpa intervensi manual dari *administrator* sistem.

c. Metode analisis *average time*

Metode *Average Time* muncul dari kebutuhan evaluasi performa pada sistem basis data terdistribusi untuk mengukur tingkat efisiensi durasi dan stabilitas sinkronisasi secara objektif. Metode ini berlandaskan pada metodologi pengujian mekanisme replikasi yang diteliti oleh Florensia dkk. (2024, hal: 33), guna memverifikasi keandalan sistem dalam menangani beban kerja data secara terus-menerus. Dengan mengacu pada Persamaan 2.1 yang telah diuraikan pada landasan teori, data durasi yang diperoleh dari hasil pengujian akan dikalkulasi dan dianalisis menggunakan pendekatan statistik deskriptif dan komparatif sebagai berikut:

1) Analisis Efisiensi Otomatisasi Backup Data

   Analisis ini bertujuan untuk melihat stabilitas sistem dalam menangani pertumbuhan volume data dari waktu ke waktu.

2) Analisis Latensi Sinkronisasi

   Mengukur selisih waktu (latensi) antara penulisan di VPS Master hingga data muncul di VPS Slave. Nilai *Seconds_Behind_Master* akan dianalisis untuk menentukan tingkat sinkronisasi yang presisi guna menghindari keterlambatan informasi medis.

3) Analisis Performa Akses

   Menganalisis *throughput* (jumlah transaksi per detik) pada arsitektur *master-slave* yang memisahkan koneksi baca dan tulis, serta menghitung persentase peningkatan kecepatan akses data setelah beban kerja didistribusikan ke VPS Slave.

3.2 Data dan Pengumpulan Data

Tahap pengumpulan data merupakan bagian krusial dalam memastikan validitas analisis terhadap sistem otomatisasi *backup* yang diimplementasikan pada jaringan *hybrid*. Proses ini melibatkan identifikasi kebutuhan data teknis melalui studi literatur dan observasi langsung terhadap objek penelitian, yakni sistem informasi pemeriksaan pasien rawat inap. Data yang dikumpulkan mencakup data primer yang diperoleh secara *real-time* melalui hasil eksperimen pada lingkungan *Virtual Private Server* (VPS) serta data sekunder pendukung untuk memperkuat landasan teori mengenai manajemen basis data dan mekanisme replikasi. Seluruh

data yang dihimpun akan difokuskan pada pengukuran efisiensi durasi pencadangan dan konsistensi sinkronisasi objek *database* guna menjawab rumusan masalah penelitian.

3.2.1 Identifikasi jenis data

Data yang dibutuhkan dalam penelitian ini dibagi menjadi dua kategori utama guna mendukung analisis *average time* dan evaluasi sistem:

a. Data Teknis Operasional (Kuantitatif):
   1) Durasi waktu yang dibutuhkan untuk setiap proses *backup* menggunakan mariadb-*backup*.
   2) Ukuran basis data (*database size*) yang dikirimkan dari jaringan *hybrid* sistem pemeriksaan pasien rawat inap.
   3) Nilai latensi sinkronisasi atau waktu tunda (*delay*) antara server *Master* dan *Slave*.
   4) *Log* keberhasilan atau kegagalan eksekusi perintah *cron job* pada Linux.

b. Data Kebutuhan Sistem (Kualitatif/Teknis):
   1) Struktur tabel dan objek *database* sistem pemeriksaan pasien yang akan direplikasi.
   2) Konfigurasi parameter pada file **my.cnf** atau **my.ini** untuk mendukung replikasi dan otomatisasi.

3.2.2 Sumber data

Penelitian ini menggunakan dua sumber data utama untuk menjamin validitas hasil pengujian:

a. Data Primer
   1) Diperoleh langsung melalui hasil pengujian atau eksperimen pada sistem yang dibangun.
   2) Data ini mencakup catatan waktu nyata (*real-time logs*) dari proses otomatisasi *backup* dan status sinkronisasi *master-slave*.

b. Data Sekunder

1) Berasal dari studi literatur, jurnal penelitian terdahulu, dan dokumentasi resmi mengenai MySQL, mariadb-*backup*, serta sistem informasi rumah sakit.
2) Data ini digunakan sebagai pembanding atau standar referensi dalam menganalisis kinerja sistem.

3.2.3 Cara pengumpulan data

Data akan dikumpulkan melalui langkah-langkah terstruktur berikut:

a. Observasi Sistem (Eksperimen)

Peneliti melakukan pengamatan langsung terhadap jalannya simulasi pengiriman data dari jaringan *hybrid* ke server *Master*. Data primer seperti status sinkronisasi dikumpulkan dengan memantau perintah **SHOW SLAVE STATUS\G;** pada MySQL.

b. Pengujian Terkontrol (*Testing*)

Menjalankan skenario otomatisasi *backup* sesuai jadwal *cron job* yang telah ditentukan. Waktu mulai dan waktu selesai setiap sesi *backup* dicatat secara otomatis ke dalam file *log* sistem untuk diolah lebih lanjut menggunakan metode *average time*.

c. Dokumentasi *Log*

Mengambil data histori dari *log* mariadb-*backup* dan *log* Linux (*syslog/cronlog*). Dokumentasi ini berfungsi untuk memverifikasi integritas data dan konsistensi otomatisasi tanpa adanya intervensi manual.

d. Studi Literatur

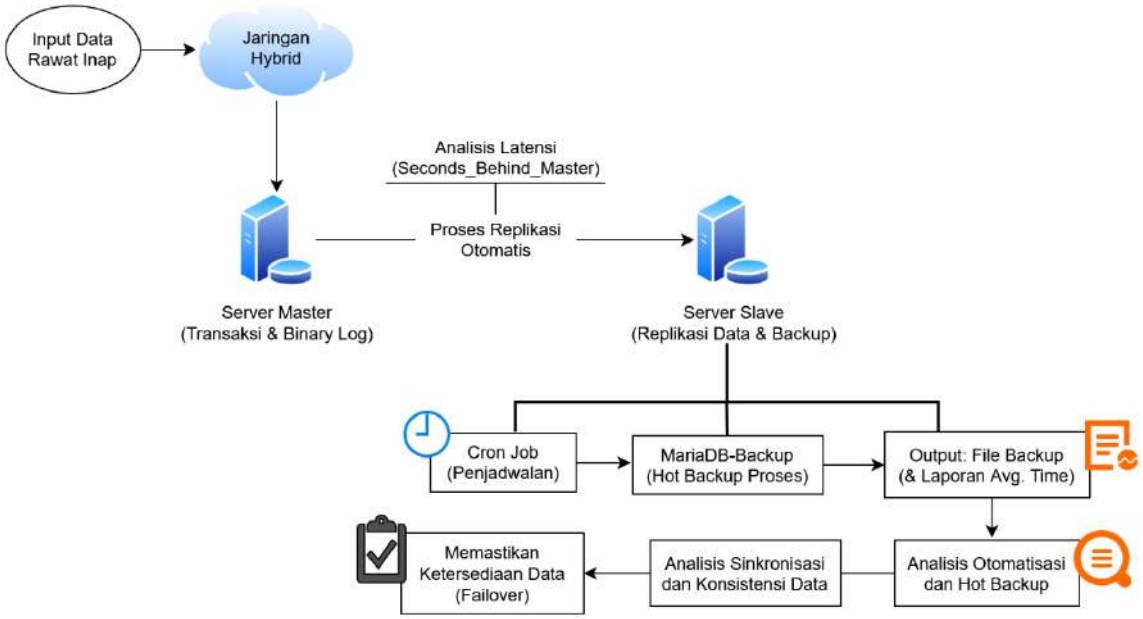
Mengumpulkan data pendukung melalui observasi pada literatur tentang sistem manajemen basis data dan sistem replikasi guna memahami bagian-bagian teknis yang harus dikonfigurasi.

3.3 Rancangan Sistem (*Hardware/Software*)

Arsitektur ini dirancang untuk beroperasi pada lingkungan jaringan *hybrid* yang menghubungkan terminal pemeriksaan pasien di unit rawat inap dengan pusat data (*data center*). Untuk rancangan sistem meliputi perangkat keras dan perangkat lunak, berikut penjelasannya:

a. Perangkat Keras (*Hardware*):
    1) Server *Master*: Server utama berbasis Linux yang berfungsi menangani transaksi data pemeriksaan pasien secara *real-time*.
    2) Server *Slave*: Server sekunder yang berfungsi sebagai replika data dan tempat dilakukannya proses otomatisasi *backup* untuk menjaga performa *Master*.
    3) Infrastruktur Jaringan: Menggunakan 2 VPS (*Virtual Private Server*) dengan spesifikasi tertentu yang sudah dikonfigurasi agar bisa saling berkomunikasi.
    4) *Client* (Terminal Rawat Inap): Perangkat komputer yang digunakan tenaga medis untuk memasukkan data pasien.
b. Perangkat Lunak (*Software*):
    1) Sistem Operasi: Linux Ubuntu Server
    2) *Database Management System:* MySQL Server dengan fitur replikasi aktif.
    3) *Backup Engine:* Mariadb-*backup* (untuk fitur *hot backup*).
    4) *Automation Tool: Cron job* (penjadwal tugas pada Linux).

Arsitektur ini mengintegrasikan jaringan *hybrid* dan otomatisasi *backup* untuk operasional rawat inap rumah sakit. Melalui replikasi *master-slave* MySQL, objek basis data disinkronkan secara otomatis guna menjamin konsistensi informasi pemeriksaan pasien rawat inap. Server *Master* melayani transaksi *real-time*, sementara server *Slave* mengelola cadangan menggunakan mariadb-*backup* dan *cron job* demi menjaga stabilitas performa server utama. Mekanisme *hot backup* ini menjamin ketersediaan data tanpa mengganggu aktivitas layanan sistem. Dapat dilihat pada gambar 3.2 berikut,



Gambar 3.2 Visualisasi Arsitektur Sistem

3.3.1 Rancangan *input*

Tahap ini menjelaskan data atau instruksi yang masuk ke dalam sistem:

a. *Input* berupa data identitas pasien, rekam medis, hasil pemeriksaan, dan tindakan medis yang dimasukkan melalui aplikasi rawat inap.
b. Parameter koneksi antara server *Master* dan *Slave* (IP *address*, *username*, *password*, dan *log-bin file*).
c. Instruksi mengenai waktu spesifik (setiap pukul 02:00 WIB) bagi sistem untuk memulai proses *backup* otomatis.

3.3.2 Rancangan proses

Menguraikan langkah-langkah teknis pengolahan data dari input menjadi cadangan yang aman:

a. Setiap data yang masuk ke server *Master* dicatat dalam *binary log* dan dikirimkan secara otomatis ke server *Slave* melalui jaringan untuk memastikan konsistensi data.
b. Kueri tulis (*read*) diarahkan ke *master*, sementara kueri baca diarahkan ke *Slave* untuk mengoptimalkan performa akses.
c. *Cron job* pada Linux secara otomatis menjalankan skrip *bash* yang memanggil perintah mariadb-*backup* sesuai jadwal.

d. Mariadb-*backup* menyalin file data fisik pada sisi *Slave* tanpa melakukan penguncian tabel (*non-blocking*), sehingga *input* data pasien pada *Master* tetap bisa berjalan secara *real-time*.

3.3.3 Rancangan *output*

Hasil akhir yang dihasilkan oleh sistem dapat dijelaskan sebagai berikut:

a. Basis data pada server *Slave* yang identik dengan *Master*, siap digunakan jika terjadi kegagalan sistem (*failover*).
b. File cadangan *database* dalam format terkompresi yang tersimpan secara aman di direktori cadangan.
c. Laporan status keberhasilan atau kegagalan proses *backup*.
d. Data numerik mengenai rata-rata waktu yang dibutuhkan sistem untuk menyelesaikan satu siklus *backup* sebagai bahan analisis performa.

3.4 Teknik Pengujian

Pengujian dilakukan dengan metode eksperimental melalui beberapa skenario simulasi beban kerja pada sistem rawat inap:

3.4.1 Pengujian sinkronisasi dan konsistensi data

Pada pengujian sinkronisasi dan konsistensi data diuraikan sebagai berikut:

a. Dilakukan input data pemeriksaan pasien secara acak pada VPS *Master* untuk memicu mekanisme replikasi.
b. Dibandingkan jumlah baris tabel dan nilai data pada VPS *Slave* segera setelah transaksi di *Master* selesai untuk memastikan konsistensi data.
c. Diamati perintah **SHOW SLAVE STATUS\G;** untuk memastikan proses *Slave_IO_Running* dan *Slave_SQL_Running* dalam kondisi aktif.

3.4.2 Pengujian otomatisasi dan *hot backup*

Untuk pengujian otomatisasi dan *hot backup* dijabarkan sebagai berikut:

a. Memverifikasi ketepatan waktu eksekusi skrip *backup* otomatis sesuai jadwal yang telah diatur dalam sistem Linux.
b. Mengukur kemampuan mariadb-*backup* dalam melakukan *hot backup* tanpa mengunci tabel (*locking*), sehingga input data pasien pada VPS *Master* tidak terhenti selama proses pencadangan berlangsung.

c. Melakukan restorasi berkala pada file *output backup* untuk memastikan data hasil otomatisasi dapat terbaca dan valid.

3.4.3 Pengujian ketersediaan (*failover*)

Rencana pengujian *failover* untuk memastikan kesinambungan layanan dijelaskan sebagai berikut:

a. Mematikan layanan MySQL pada VPS *Master* secara mendadak untuk menguji ketersediaan data pada VPS *Slave*.
b. Memastikan sistem informasi rawat inap tetap dapat melakukan kueri baca (*read*) menggunakan data di server *Slave* meskipun server utama sedang tidak tersedia.

BAB IV
HASIL DAN PEMBAHASAN

4.1 Hasil Implementasi Sistem

Bagian ini memaparkan hasil implementasi teknis yang mencakup infrastruktur jaringan *hybrid*, konfigurasi basis data, dan mekanisme otomatisasinya. Melalui dokumentasi konfigurasi, seluruh parameter teknis dianalisis untuk memastikan replikasi *master-slave* dan skema *backup* berjalan sesuai rancangan.

4.1.1 Realisasi lingkungan kerja

Pada tahap awal implementasi teknis, realisasi lingkungan kerja (*environment*) berupa infrastruktur jaringan *hybrid* yang mengintegrasikan terminal lokal rumah sakit dengan pusat data berbasis *cloud*. Lingkungan produksi (*production-grade*) ini dibangun menggunakan 2 (dua) unit *Virtual Private Server* (VPS) Linux Ubuntu Server yang berfungsi secara spesifik sebagai server *Master* dan server *Slave*. Realisasi spesifikasi perangkat keras logis (*virtual hardware*) untuk kedua server tersebut dipaparkan secara rinci pada tabel 4.1 berikut.

Tabel 4.1 Spesifikasi Teknis *Virtual Private Server* (VPS)

Komponen Sistem	Spesifikasi VPS *Master*	Spesifikasi VPS *Slave*
Sistem Operasi	Linux Ubuntu Server 22.04	Linux Ubuntu Server 22.04
Arsitektur CPU	1 Core vCPU	1 Core vCPU
Kapasitas RAM	1 GB RAM	1 GB RAM
Penyimpanan (Storage)	60 GB NVMe SSD	60 GB NVMe SSD
Konektivitas Jaringan	IP Publik Statis	IP Publik Statis

Dilakukan konfigurasi jaringan pada tingkat server agar kedua VPS dapat saling berkomunikasi secara aman melalui protokol TCP/IP.

4.1.2 Hasil konfigurasi replikasi *master-slave* MySQL

Setelah topologi jaringan *hybrid* berhasil direalisasikan, tahapan selanjutnya adalah mengonfigurasi sistem replikasi basis data *master-slave* menggunakan MariaDB/MySQL, guna memastikan setiap perubahan data rawat inap pada VPS *Master* tersinkronisasi ke VPS *Slave*.

Pada VPS *Master*, konfigurasi berkas **50-server.cnf** dilakukan dengan mengaktifkan *binary logging* (*log*_bin) dan menetapkan **server-id = 1** untuk mencatat seluruh transaksi DML (*Insert, Update, Delete*), dapat dilihat pada gambar 4.1 berikut.

```
server-id               = 1
log_bin                 = /var/log/mysql/mysql-bin.log
```

Gambar 4.1 Penetapan server-id dan *log*_bin *Node Master*

Pada VPS *Slave*, ditetapkan **server-id = 2** dengan pointer replikasi yang diarahkan ke IP publik statis VPS *Master* menggunakan kredensial khusus, dapat dilihat pada gambar 4.2 berikut.

```
server-id               = 2
log_bin                 = /var/log/mysql/mysql-bin.log
```

Gambar 4.2 Penetapan server-id dan *log*_bin *Node Slave*

Setelah konfigurasi file **50-server.cnf** dilakukan pada masing-masing *node*, selanjutnya masuk ke MariaDB pada sisi *node master* untuk memberikan akses replikasi pada *node slave* melalui contoh baris perintah **GRANT REPLICATION SLAVE ON *.* TO 'user'@'IP Slave' IDENTIFIED BY 'password'**.

Untuk melihat informasi status dari replikasi *node master* dengan memasukkan baris perintah **SHOW MASTER STATUS;** untuk melihat nama file *binary log* dan posisi *log* replikasi yang akan dimasukkan pada sisi replikasi *node slave* agar proses sinkronisasi antar database berjalan akurat dari titik koordinat tersebut, dapat dilihat pada gambar 4.3 berikut.

```
MariaDB [(none)]> SHOW MASTER STATUS;
+------------------+----------+--------------+------------------+
| File             | Position | Binlog_Do_DB | Binlog_Ignore_DB |
+------------------+----------+--------------+------------------+
| mysql-bin.000003 |   181952 |              | phpmyadmin       |
+------------------+----------+--------------+------------------+
1 row in set (0.002 sec)
```

Gambar 4.3 Status Replikasi pada *Node Master*

Ketika informasi sudah didapatkan dari status *node master* tersebut, selanjutnya masuk ke MariaDB pada sisi *node slave* untuk melakukan konfigurasi sinkronisasi replikasi dari informasi file *binary log* dan posisi *log* yang akan digunakan, contoh baris sintaksnya dapat dilihat pada gambar 4.4 berikut

```
change master to
   -> master_host='Alamat IP Master',
   -> master_user='user',
   -> master_password='password',
   -> master_port=3306,
   -> master_log_file='mysql-bin.000003',
   -> master_log_pos=181952;
```

Gambar 4.4 Baris Sintaks Konfigurasi Replikasi *Node Slave*

Keberhasilan replikasi diverifikasi melalui kueri **SHOW SLAVE STATUS\G;** dengan hasil parameter kritis disajikan pada tabel 4.2 berikut.

Tabel 4.2 Parameter Kritis Status Replikasi pada *Node Slave*

Parameter Replikasi	**Nilai Status (*Output*)**	**Indikasi Sistem**
Slave_IO_Running	**Yes**	Terhubung ke Master dan membaca *Binary Log*
Slave_SQL_Running	**Yes**	Berhasil mengeksekusi kueri dari *Master* ke Database *Slave*
Master_Log_File	mysql-bin.000003	Berkas *log* aktif yang sedang dibaca saat ini
Read_Master_Log_Pos	181952	Posisi pointer pembacaan baris *log* terakhir

Nilai *Yes* pada parameter *Slave_IO_Running* dan *Slave_SQL_Running* di tabel 4.2 membuktikan bahwa *Thread* I/O *node slave* berhasil terhubung ke *node master*, membaca *binary log*, dan menuliskan instruksi data ke penyimpanan lokal tanpa error.

4.1.3 Hasil implementasi otomatisasi *backup* terjadwal

Setelah replikasi *master-slave* dipastikan berjalan stabil, kemudian diimplementasikan sistem otomatisasi backup pada VPS *slave* dengan mengintegrasikan mariadb-*backup* sebagai mesin pencadangan utama dan *cron job* Linux Ubuntu Server sebagai penjadwal tugas. Dua skrip bash disusun untuk keperluan tersebut, yakni mysql_*full_backup*.sh untuk pencadangan penuh dan mysql_*incremental_backup*.sh untuk pencadangan inkremental, keduanya memanfaatkan kemampuan *hot backup* mariadb-*backup* yang dapat menyalin berkas data fisik tanpa penguncian tabel, untuk kedua skripnya dapat dilihat pada gambar 4.5 dan gambar 4.6 berikut.

```
DATE=$(date +%F)
BASE_DIR="/backup/mysql/${DATE}"
FULL_DIR="${BASE_DIR}/full"
LOG_FILE="/var/log/mariadb_backup.log"

log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" >> "$LOG_FILE"
}

log "====== MULAI FULL BACKUP ======"

# Buat direktori
mkdir -p "$FULL_DIR"

# Jalankan full backup
if mariabackup --backup \
    --target-dir="$FULL_DIR" \
    >> "$LOG_FILE" 2>&1; then

    # Prepare full backup agar siap jadi base incremental
    mariabackup --prepare \
        --apply-log-only \
        --target-dir="$FULL_DIR" \
        >> "$LOG_FILE" 2>&1

    log "Full backup BERHASIL: $FULL_DIR"
else
    log "Full backup GAGAL!"
    exit 1
fi

# Hapus backup lebih dari 7 hari
find /backup/mysql -mindepth 1 -maxdepth 1 -type d -mtime +7 -exec rm -rf {} \;
log "Cleanup backup lama selesai"
log "====== SELESAI FULL BACKUP ======"
```

Gambar 4.5 Skrip mysql_full_backup.sh

Sementara itu, skrip kedua yang dirancang secara spesifik untuk menangani proses penyalinan perubahan data secara inkremental disajikan pada gambar 4.6 berikut.

```
DATE=$(date +%F)
HOUR=$(date +%H)
BASE_DIR="/backup/mysql/${DATE}"
FULL_DIR="${BASE_DIR}/full"
LOG_FILE="/var/log/mariadb_backup.log"

log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" >> "$LOG_FILE"
}

log "====== MULAI INCREMENTAL BACKUP (jam ${HOUR}) ======"

# Tentukan nama folder target berdasarkan jam
case "$HOUR" in
    09) TARGET_DIR="${BASE_DIR}/inc-09"
        ;;
    15) TARGET_DIR="${BASE_DIR}/inc-15"
        ;;
    21) TARGET_DIR="${BASE_DIR}/inc-21"
        ;;
    *)  log "Jam tidak dikenal: $HOUR. Backup dibatalkan."
        exit 1
        ;;
esac

# Tentukan base incremental (dari mana perubahan dihitung)
# Urutan prioritas: inc-15 > inc-09 > full
if   [ -d "${BASE_DIR}/inc-15" ] && [ "$HOUR" == "21" ]; then
    BASE_INC="${BASE_DIR}/inc-15"
elif [ -d "${BASE_DIR}/inc-09" ] && [ "$HOUR" == "15" ]; then
    BASE_INC="${BASE_DIR}/inc-09"
else
    BASE_INC="$FULL_DIR"
fi
# Validasi: full backup harus sudah ada
if [ ! -d "$FULL_DIR" ]; then
    log "ERROR: Full backup tidak ditemukan di $FULL_DIR"
    log "Jalankan full backup terlebih dahulu!"
    exit 1
fi

# Buat direktori target
mkdir -p "$TARGET_DIR"

log "Base incremental: $BASE_INC"
log "Target: $TARGET_DIR"

# Jalankan incremental backup
if mariabackup --backup \
    --target-dir="$TARGET_DIR" \
    --incremental-basedir="$BASE_INC" \
    >> "$LOG_FILE" 2>&1; then

    log "Incremental backup BERHASIL: $TARGET_DIR"
else
    log "Incremental backup GAGAL!"
    exit 1
fi

log "====== SELESAI INCREMENTAL BACKUP (jam ${HOUR}) ======"
```

Gambar 4.6 Skrip mysql_incremental_backup.sh

Seluruh hasil cadangan disimpan pada direktori /*backup*/mysql dan dijadwalkan melalui crontab -e, *full backup* dieksekusi setiap pukul 02.00 WIB pada periode lalu lintas data terendah, sedangkan *incremental backup* dijalankan pada pukul 09.00, 15.00, dan 21.00 WIB. Konfigurasi baris perintah *crontab* yang berhasil diimplementasikan dipaparkan pada tabel 4.3 berikut.

Tabel 4.3 Konfigurasi Penjadwalan *Crontab* pada Linux *Node Slave*

No	Sintaks *Crontab*	Representasi Waktu	Aksi / Skrip yang Dieksekusi	Jenis Operasi Backup
1	0 2 * * *	Setiap hari pukul 02:00 WIB	/usr/local/bin/mysql_full_backup.sh	*Full backup* (Salinan Fisik Basis Data Utuh)
2	0 9 * * *	Setiap hari pukul 09:00 WIB	/usr/local/bin/mysql_incremental_backup.sh	*Incremental backup* (Salinan Perubahan Data)
3	0 15 * * *	Setiap hari pukul 15:00 WIB	/usr/local/bin/mysql_incremental_backup.sh	*Incremental backup* (Salinan Perubahan Data)
4	0 21 * * *	Setiap hari pukul 21:00 WIB	/usr/local/bin/mysql_incremental_backup.sh	*Incremental backup* (Salinan Perubahan Data)

Berkas cadangan yang dihasilkan memiliki penamaan terstruktur berbasis tanggal (*timestamping*), yang mempermudah proses identifikasi apabila sewaktu-waktu diperlukan pemulihan data (*data recovery*), contohnya dapat dilihat pada gambar 4.7 dan gambar 4.8 berikut.

```
[redacted]# ls -lh /backup/mysql
total 36K
drwxr-xr-x 6 root root 4.0K Jun  6 21:00 2026-06-06
drwxr-xr-x 6 root root 4.0K Jun  7 21:00 2026-06-07
drwxr-xr-x 6 root root 4.0K Jun  8 21:00 2026-06-08
drwxr-xr-x 6 root root 4.0K Jun  9 21:00 2026-06-09
drwxr-xr-x 6 root root 4.0K Jun 10 21:00 2026-06-10
drwxr-xr-x 6 root root 4.0K Jun 11 21:00 2026-06-11
drwxr-xr-x 6 root root 4.0K Jun 12 21:00 2026-06-12
drwxr-xr-x 6 root root 4.0K Jun 13 21:00 2026-06-13
drwxr-xr-x 5 root root 4.0K Jun 14 15:00 2026-06-14
```

Gambar 4.7 Direktori Hasil *Backup* pada *Node Slave*

Adapun rincian struktur dan isi berkas yang terdapat di dalam salah satu direktori pencadangan harian tersebut dapat dilihat secara lebih jelas pada gambar 4.8 berikut.

```
[redacted] ls -lh /backup/mysql/2026-06-14
total 16K
drwxr-xr-x 9 root root 4.0K Jun 14 02:00 full
drwxr-xr-x 9 root root 4.0K Jun 14 09:00 inc-09
drwxr-xr-x 9 root root 4.0K Jun 14 15:00 inc-15
drwxr-xr-x 9 root root 4.0K Jun 14 21:00 inc-21
[redacted] ls -lh /backup/mysql/2026-06-14/full
total 125M
-rw-r----- 1 root root 416K Jun 14 02:00 aria_log.00000001
-rw-r----- 1 root root   52 Jun 14 02:00 aria_log_control
-rw-r----- 1 root root  285 Jun 14 02:00 backup-my.cnf
-rw-r----- 1 root root  49M Jun 14 02:00 ib_logfile0
-rw-r----- 1 root root  76M Jun 14 02:00 ibdata1
drwx------ 2 root root 4.0K Jun 14 02:00 monitor
drwx------ 2 root root 4.0K Jun 14 02:00 mysql
drwx------ 2 root root 4.0K Jun 14 02:00 performance_schema
drwx------ 2 root root 4.0K Jun 14 02:00 phpmyadmin
drwx------ 2 root root 4.0K Jun 14 02:00 politeknik
drwx------ 2 root root 4.0K Jun 14 02:00 rawat_inap_vps
drwx------ 2 root root  12K Jun 14 02:00 sys
-rw-r----- 1 root root   35 Jun 14 02:00 xtrabackup_binlog_info
-rw-r----- 1 root root  107 Jun 14 02:00 xtrabackup_checkpoints
-rw-r----- 1 root root  543 Jun 14 02:00 xtrabackup_info
```

Gambar 4.8 Isi dari Folder Hasil *Backup* pada 14 Juni 2026

Dengan keberhasilan implementasi otomatisasi *backup* terjadwal ini, telah mewujudkan mekanisme pencadangan data rekam medis pasien rawat inap yang berjalan di latar belakang (*background process*) secara konsisten, efisien, dan selaras dengan standar tata kelola infrastruktur data rumah sakit.

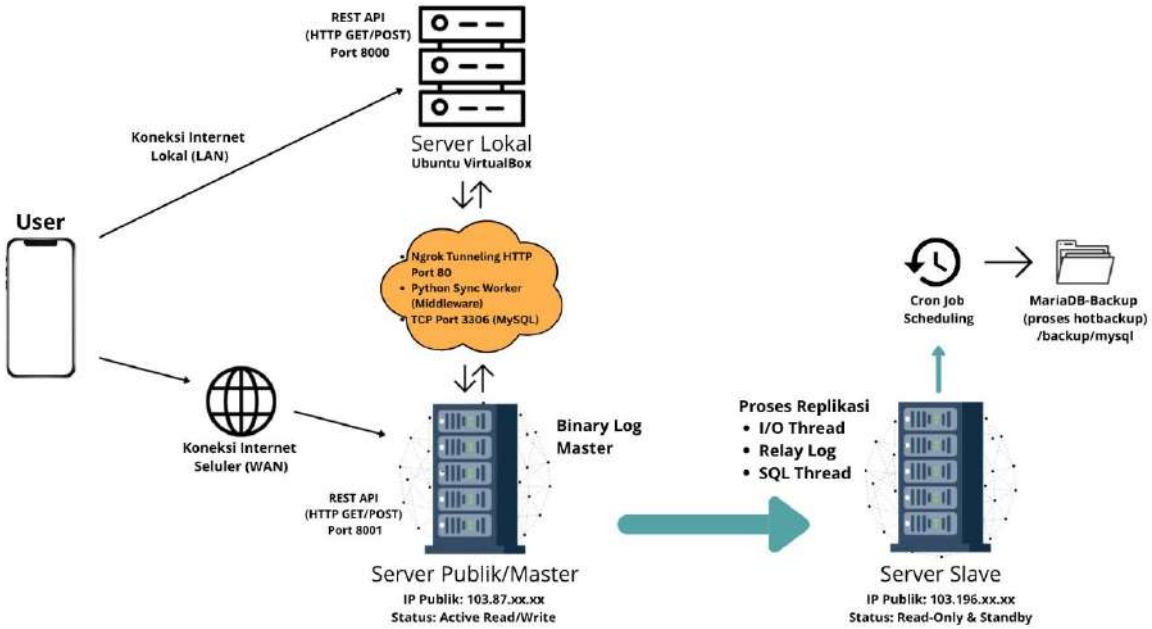
4.2 Pengujian dan Analisis Data

Subbab ini menyajikan hasil pengujian dan analisis performa serta keandalan sistem otomatisasi *backup* data rekam medis. Melalui injeksi data pada jaringan *hybrid*, evaluasi difokuskan pada efektivitas replikasi *master-slave* MySQL,

efisiensi mariadb-*backup*, dan ketepatan *cron job*. Analisis ini bertujuan memastikan sinkronisasi, konsistensi, dan integritas data antara *node master* dan *node slave* terjaga secara optimal.

4.2.1 Arsitektur jaringan *hybrid* dan *master-slave*

Bagian ini menyajikan rancangan arsitektur sistem yang mengintegrasikan infrastruktur jaringan *hybrid* dengan topologi basis data *master-slave*. Penjelasan ini menjadi landasan teknis untuk memahami jalur transmisi data rekam medis dari terminal lokal menuju *node master*, hingga proses propagasi otomatis ke *node slave* sebagai pelaksana cadangan data harian, dilihat pada gambar 4.9 berikut.



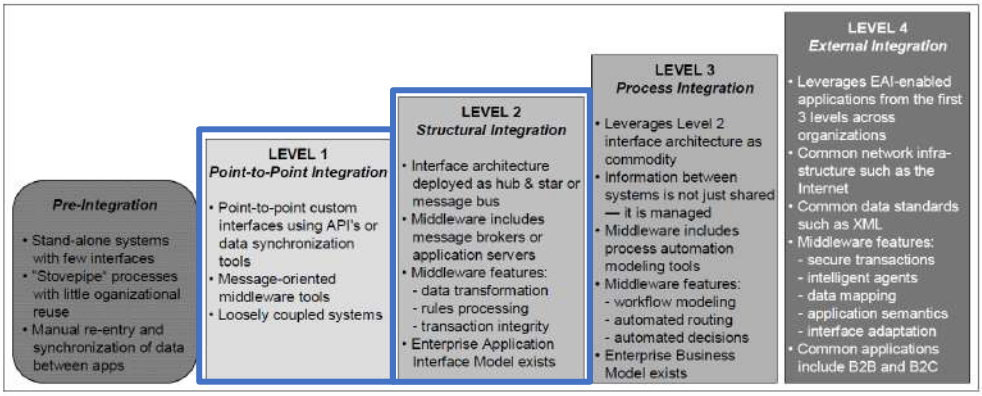
Gambar 4.9 Alur Sistem Jaringan *Hybrid* dan *master-slave*

Berikut adalah penjelasan mengenai alur operasional sistem jaringan *hybrid* dan arsitektur *master-slave* sesuai dengan visualisasi arsitektur gambar 4.8 tersebut:

a. Pengguna mengakses aplikasi melalui jalur lokal (LAN/Port 8000) atau publik (WAN/Port 8001), yang keduanya terintegrasi ke dalam jaringan *hybrid* menggunakan Ngrok *Tunneling* dan *Python Sync Worker* untuk sinkronisasi basis data (MySQL/Port 3306).
b. Transaksi *read/write* diproses secara *real-time* pada *node master* melalui *Binary Log*, lalu direplikasi secara otomatis oleh *node slave* melalui mekanisme *I/O Thread*, *Relay Log*, dan *SQL Thread*.

c. Tugas *hot backup* diisolasi sepenuhnya pada *node slave* menggunakan *cron job* dan mariadb-*backup* untuk menghasilkan berkas cadangan otomatis di dalam direktori /*backup*/mysql.

Evaluasi kematangan infrastruktur dalam mengintegrasikan sistem basis data dapat diukur menggunakan instrumen *Integration Maturity Model* (Model Kematangan Integrasi) gagasan Schmidt. Kerangka kerja ini mengklasifikasikan evolusi sistem secara hierarkis, mulai dari fase praintegrasi yang terisolasi hingga tahap integrasi eksternal antar-organisasi. Pemetaan tersebut menjadi panduan strategis untuk menilai stabilitas operasional arsitektur terdistribusi, termasuk skema replikasi pada jaringan *hybrid*. Visualisasi tahapan model kematangan integrasi aplikasi ini diilustrasikan pada gambar 4.10 berikut.



Gambar 4.10 Model Kematangan Schmidt untuk Integrasi Aplikasi
(Sumber: Mendoza dkk., 2003, hal: 2)

Berdasarkan 4 level *Integration Maturity Model* Schmidt pada tahun 2000 seperti pada gambar 4.10 diatas, arsitektur replikasi *master-slave* yang dibangun secara fungsional memenuhi karakteristik **Level 1: *Point-to-Point Integration*** melalui distribusi data langsung via propagasi *binary log*. Namun, capaian tingkat data yang konsisten dan jaminan integritas transaksi antar server di lingkungan *cloud* membuktikan bahwa infrastruktur jaringan *hybrid* ini telah meletakkan landasan kuat menuju **Level 2: *Structural Integration***. Keberhasilan otomatisasi mariadb-*backup* dan *cron job* ini memastikan rumah sakit memiliki infrastruktur teknologi yang tangguh sebagai salah satu Faktor Penentu Keberhasilan (*Critical Success Factors*) strategis integrasi sistem (Mendoza dkk., 2003, hal: 3).

4.2.2 Pengujian sinkronisasi dan konsistensi data pada replikasi

Pengujian tersebut disimulasikan dengan menginjeksi minimal **50 baris data (*records*)** pemeriksaan pasien rawat inap dari jaringan *hybrid* ke dalam basis data utama pada *node master*. Segera setelah proses injeksi selesai, dilakukan verifikasi silang (*cross-validation*) secara langsung pada kedua server. Langkah pengujian ini dilakukan melalui dua tahapan pembuktian:

a. Memastikan parameter *Slave_IO_Running* dan *Slave_SQL_Running* tetap berada pada status *Yes* selama proses transaksi berlangsung, guna membuktikan tidak adanya kegagalan pipa komunikasi (*replication pipeline error*) akibat beban penulisan data kesehatan yang masuk.
b. Mengeksekusi kueri agregasi **SELECT COUNT(*)** serta memeriksa kesamaan nilai *Primary Key* pada tabel-tabel kritis database pemeriksaan pasien rawat inap antara *node master* dan *node slave*.

Hasil komparasi struktural dan jumlah baris data yang diperoleh dari arsitektur terdistribusi ini disajikan pada tabel 4.4 berikut.

Tabel 4.4 Komparasi Jumlah Baris dan Konsistensi Objek Basis Data

No	Nama Tabel Basis Data	Jumlah Baris (Master)	Jumlah Baris (Slave)	Selisih (Delta)	Status Konsistensi
1	**pasien**	50	50	0	Konsisten
2	**users**	6	6	0	Konsisten
3	**visit**	50	50	0	Konsisten

Berdasarkan observasi pada tabel 4.4, hasil pengujian menunjukkan nilai selisih (*delta*) sebesar 0 (nol) pada seluruh tabel yang diamati. Hal ini membuktikan bahwa setiap paket data rekam medis pasien yang dituliskan pada *node master* telah diteruskan oleh *Binary Log* dan dieksekusi kembali secara sempurna oleh *SQL Thread* pada *node slave,* dapat dilihat pada gambar 4.11 dan gambar 4.12 berikut.

```
MariaDB [rawat_inap_vps]> SELECT COUNT(*) FROM pasien;
+----------+
| COUNT(*) |
+----------+
|       50 |
+----------+
1 row in set (0.002 sec)

MariaDB [rawat_inap_vps]> SELECT COUNT(*) FROM users;
+----------+
| COUNT(*) |
+----------+
|        6 |
+----------+
1 row in set (0.001 sec)

MariaDB [rawat_inap_vps]> SELECT COUNT(*) FROM visit;
+----------+
| COUNT(*) |
+----------+
|       50 |
+----------+
1 row in set (0.001 sec)
```

Gambar 4.11 Kueri Validasi Baris Data pada *Node Master*

Sebagai bentuk verifikasi silang terhadap kueri peladen utama di atas, hasil eksekusi kueri untuk memvalidasi kesamaan jumlah baris data yang telah direplikasi pada *node slave* disajikan pada gambar 4.12 berikut.

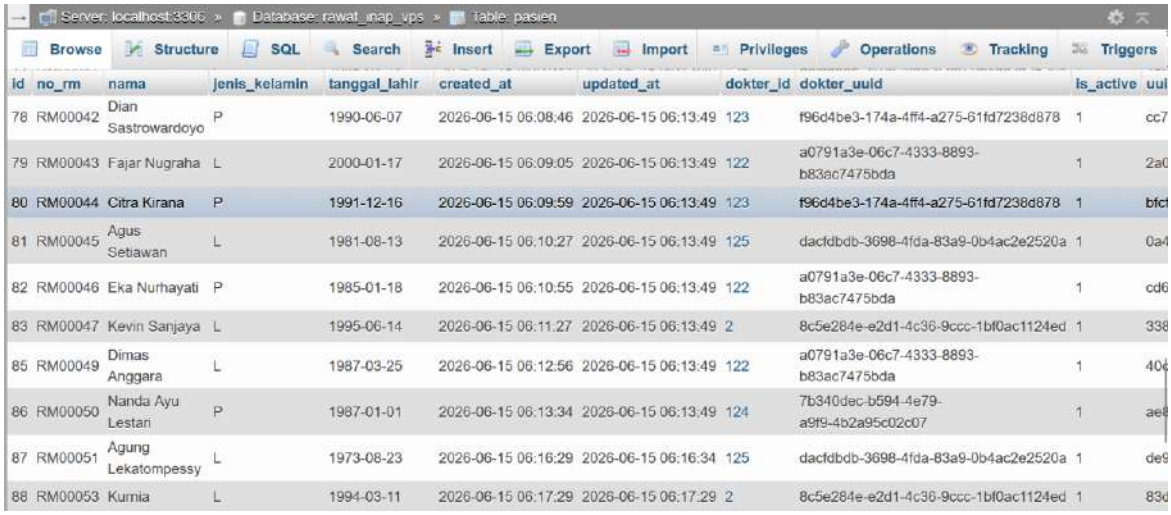
```
MariaDB [rawat_inap_vps]> SELECT COUNT(*) FROM pasien;
+----------+
| COUNT(*) |
+----------+
|       50 |
+----------+
1 row in set (0.051 sec)

MariaDB [rawat_inap_vps]> SELECT COUNT(*) FROM users;
+----------+
| COUNT(*) |
+----------+
|        6 |
+----------+
1 row in set (0.006 sec)

MariaDB [rawat_inap_vps]> SELECT COUNT(*) FROM visit;
+----------+
| COUNT(*) |
+----------+
|       50 |
+----------+
1 row in set (0.001 sec)
```

Gambar 4.12 Kueri Validasi Baris Data pada *Node Slave*

Berikut ini merupakan beberapa potongan data yang di replikasi oleh *master* dan *slave*, disajikan menggunakan baris tabel phpMyAdmin untuk mempermudah penyajian data yang beragam.



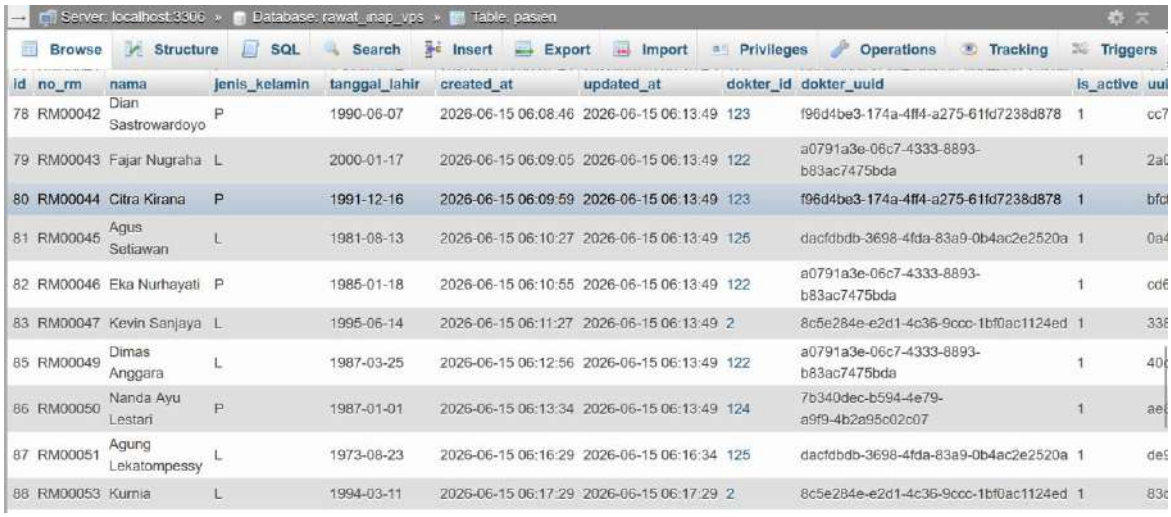
Server: localhost:3306 » Database: rawat_inap_vps » Table: pasien

Browse | Structure | SQL | Search | Insert | Export | Import | Privileges | Operations | Tracking | Triggers

id	no_rm	nama	jenis_kelamin	tanggal_lahir	created_at	updated_at	dokter_id	dokter_uuid	is_active	uu
78	RM00042	Dian Sastrowardoyo	P	1990-06-07	2026-06-15 06:08:46	2026-06-15 06:13:49	123	f96d4be3-174a-4ff4-a275-61fd7238d878	1	cc7
79	RM00043	Fajar Nugraha	L	2000-01-17	2026-06-15 06:09:05	2026-06-15 06:13:49	122	a0791a3e-06c7-4333-8893-b83ac7475bda	1	2a0
80	RM00044	Citra Kirana	P	1991-12-16	2026-06-15 06:09:59	2026-06-15 06:13:49	123	f96d4be3-174a-4ff4-a275-61fd7238d878	1	bfc
81	RM00045	Agus Setiawan	L	1981-08-13	2026-06-15 06:10:27	2026-06-15 06:13:49	125	dacfdbdb-3698-4fda-83a9-0b4ac2e2520a	1	0a4
82	RM00046	Eka Nurhayati	P	1985-01-18	2026-06-15 06:10:55	2026-06-15 06:13:49	122	a0791a3e-06c7-4333-8893-b83ac7475bda	1	cd6
83	RM00047	Kevin Sanjaya	L	1995-06-14	2026-06-15 06:11:27	2026-06-15 06:13:49	2	8c5e284e-e2d1-4c36-9ccc-1bf0ac1124ed	1	338
85	RM00049	Dimas Anggara	L	1987-03-25	2026-06-15 06:12:56	2026-06-15 06:13:49	122	a0791a3e-06c7-4333-8893-b83ac7475bda	1	406
86	RM00050	Nanda Ayu Lestari	P	1987-01-01	2026-06-15 06:13:34	2026-06-15 06:13:49	124	7b340dec-b594-4e79-a9f9-4b2a95c02c07	1	ae8
87	RM00051	Agung Lekatompessy	L	1973-08-23	2026-06-15 06:16:29	2026-06-15 06:16:34	125	dacfdbdb-3698-4fda-83a9-0b4ac2e2520a	1	de9
88	RM00053	Kurnia	L	1994-03-11	2026-06-15 06:17:29	2026-06-15 06:17:29	2	8c5e284e-e2d1-4c36-9ccc-1bf0ac1124ed	1	83c

Gambar 4.13 Potongan Baris Data pada Tabel pasien *Node Master*

Dilihat pada gambar 4.13 diatas menyajikan tampilan antarmuka pengelolaan basis data MySQL yang memuat contoh rincian rekaman identitas medis pasien pada tabel **pasien** sisi *node master* di dalam sistem rawat inap.



Server: localhost:3306 » Database: rawat_inap_vps » Table: pasien

Browse | Structure | SQL | Search | Insert | Export | Import | Privileges | Operations | Tracking | Triggers

id	no_rm	nama	jenis_kelamin	tanggal_lahir	created_at	updated_at	dokter_id	dokter_uuid	is_active	uu
78	RM00042	Dian Sastrowardoyo	P	1990-06-07	2026-06-15 06:08:46	2026-06-15 06:13:49	123	f96d4be3-174a-4ff4-a275-61fd7238d878	1	cc7
79	RM00043	Fajar Nugraha	L	2000-01-17	2026-06-15 06:09:05	2026-06-15 06:13:49	122	a0791a3e-06c7-4333-8893-b83ac7475bda	1	2a0
80	RM00044	Citra Kirana	P	1991-12-16	2026-06-15 06:09:59	2026-06-15 06:13:49	123	f96d4be3-174a-4ff4-a275-61fd7238d878	1	bfc
81	RM00045	Agus Setiawan	L	1981-08-13	2026-06-15 06:10:27	2026-06-15 06:13:49	125	dacfdbdb-3698-4fda-83a9-0b4ac2e2520a	1	0a4
82	RM00046	Eka Nurhayati	P	1985-01-18	2026-06-15 06:10:55	2026-06-15 06:13:49	122	a0791a3e-06c7-4333-8893-b83ac7475bda	1	cd6
83	RM00047	Kevin Sanjaya	L	1995-06-14	2026-06-15 06:11:27	2026-06-15 06:13:49	2	8c5e284e-e2d1-4c36-9ccc-1bf0ac1124ed	1	338
85	RM00049	Dimas Anggara	L	1987-03-25	2026-06-15 06:12:56	2026-06-15 06:13:49	122	a0791a3e-06c7-4333-8893-b83ac7475bda	1	406
86	RM00050	Nanda Ayu Lestari	P	1987-01-01	2026-06-15 06:13:34	2026-06-15 06:13:49	124	7b340dec-b594-4e79-a9f9-4b2a95c02c07	1	ae8
87	RM00051	Agung Lekatompessy	L	1973-08-23	2026-06-15 06:16:29	2026-06-15 06:16:34	125	dacfdbdb-3698-4fda-83a9-0b4ac2e2520a	1	de9
88	RM00053	Kurnia	L	1994-03-11	2026-06-15 06:17:29	2026-06-15 06:17:29	2	8c5e284e-e2d1-4c36-9ccc-1bf0ac1124ed	1	83c

Gambar 4.14 Potongan Baris Data pada Tabel pasien *Node Slave*

Dilihat pada gambar 4.14 diatas menyajikan tampilan antarmuka pengelolaan basis data MySQL yang memuat contoh rincian rekaman identitas medis pasien pada tabel **pasien** sisi *node slave* di dalam sistem rawat inap.



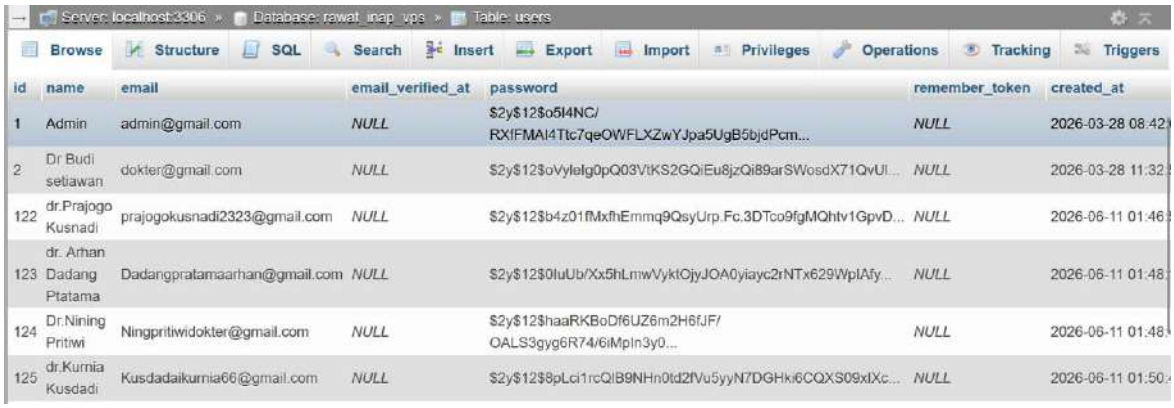
Server: localhost:3306 » Database: rawat_inap_vps » Table: users

Browse | Structure | SQL | Search | Insert | Export | Import | Privileges | Operations | Tracking | Triggers

id	name	email	email_verified_at	password	remember_token	created_at
1	Admin	admin@gmail.com	NULL	$2y$12$o5I4NC/ RXfFMAI4Ttc7qeOWFLXZwYJpa5UgB5bjdPcm...	NULL	2026-03-28 08:42:
2	Dr Budi setiawan	dokter@gmail.com	NULL	$2y$12$oVyIeIg0pQ03VtKS2GQiEu8jzQi89arSWosdX71QvUl...	NULL	2026-03-28 11:32:
122	dr.Prajogo Kusnadi	prajogokusnadi2323@gmail.com	NULL	$2y$12$b4z01fMxfhEmmq9QsyUrp.Fc.3DTco9fgMQhtv1GpvD...	NULL	2026-06-11 01:46:
123	dr. Arhan Dadang Ptatama	Dadangpratamaarhan@gmail.com	NULL	$2y$12$0luUb/Xx5hLmwVyktOjyJOA0yiayc2rNTx629WplAfy...	NULL	2026-06-11 01:48:
124	Dr.Nining Pritiwi	Ningpritiwidokter@gmail.com	NULL	$2y$12$haaRKBoDf6UZ6m2H6fJF/ OALS3gyg6R74/6iMpIn3y0...	NULL	2026-06-11 01:48:
125	dr.Kurnia Kusdadi	Kusdadaikurnia66@gmail.com	NULL	$2y$12$8pLci1rcQIB9NHn0td2fVu5yyN7DGHki6CQXS09xIXc...	NULL	2026-06-11 01:50:

Gambar 4.15 Potongan Baris Data pada Tabel users *Node Master*

Dapat diamati pada gambar 4.15 diatas menyajikan tampilan antarmuka pengelolaan basis data MySQL yang memuat contoh rincian identitas *login* dokter dan *administrator* pada tabel **users** sisi *node master* di dalam sistem rawat inap.



Server: localhost:3306 » Database: rawat_inap_vps » Table: users

Browse | Structure | SQL | Search | Insert | Export | Import | Privileges | Operations | Tracking | Triggers

id	name	email	email_verified_at	password	remember_token	created_at
1	Admin	admin@gmail.com	NULL	$2y$12$o5I4NC/ RXfFMAI4Ttc7qeOWFLXZwYJpa5UgB5bjdPcm...	NULL	2026-03-28 08:42:
2	Dr Budi setiawan	dokter@gmail.com	NULL	$2y$12$oVyIeIg0pQ03VtKS2GQiEu8jzQi89arSWosdX71QvUl...	NULL	2026-03-28 11:32:
122	dr.Prajogo Kusnadi	prajogokusnadi2323@gmail.com	NULL	$2y$12$b4z01fMxfhEmmq9QsyUrp.Fc.3DTco9fgMQhtv1GpvD...	NULL	2026-06-11 01:46:
123	dr. Arhan Dadang Ptatama	Dadangpratamaarhan@gmail.com	NULL	$2y$12$0luUb/Xx5hLmwVyktOjyJOA0yiayc2rNTx629WplAfy...	NULL	2026-06-11 01:48:
124	Dr.Nining Pritiwi	Ningpritiwidokter@gmail.com	NULL	$2y$12$haaRKBoDf6UZ6m2H6fJF/ OALS3gyg6R74/6iMpIn3y0...	NULL	2026-06-11 01:48:
125	dr.Kurnia Kusdadi	Kusdadaikurnia66@gmail.com	NULL	$2y$12$8pLci1rcQIB9NHn0td2fVu5yyN7DGHki6CQXS09xIXc...	NULL	2026-06-11 01:50:

Gambar 4.16 Potongan Baris Data pada Tabel users *Node Slave*

Dilihat pada gambar 4.16 diatas menyajikan tampilan antarmuka pengelolaan basis data MySQL yang memuat contoh rincian identitas *login* dokter dan *administrator* pada tabel **users** sisi *node slave* di dalam sistem rawat inap.


Server: localhost:3306 » Database: rawat_inap_vps » Table: visit

Browse | Structure | SQL | Search | Insert | Export | Import | Privileges | Operations | Tracking | Triggers

id	pasien_id	keluhan	pemeriksaan_fisik	diagnosa	tindakan	created_at	updated_at	uuid
75	67	demam, pilek dan badan terasa pegal pegal	NULL	influenza ringan	terapi simptomatik dan istirahat yang cukup	2026-06-15 09:11:18	2026-06-15 09:11:18	673fe931-57d7-4fb5-a5c9-e8243b44c80c
76	77	pegal diseluruh tubuh setelah aktivitas berat	NULL	Myalgia	pemberian analgesik dan istirahat	2026-06-15 09:11:59	2026-06-15 09:11:59	ba5c675b-4683-4579-90d4-522aab8f6e5
77	81	kontrol tekanan darah bulanan	NULL	Hipertensi ringan	evaluasi terapi dan edukasi untuk pola hidup sehat	2026-06-15 09:12:45	2026-06-15 09:12:45	2805ed7e-c7f6-4e76-904b-1fe914c885e9
78	87	badan lemas dan nafsu makan menurun	NULL	fatigue ringan	pemberian vitamin dan edukasi untuk menjaga pola hidup sehat	2026-06-15 09:13:26	2026-06-15 09:13:26	41b9e40f-b248-4d47-8e1f-e0228ba49141

Gambar 4.17 Potongan Baris Data pada Tabel visit *Node Master*

Pada gambar 4.17 diatas menyajikan tampilan antarmuka pengelolaan basis data MySQL yang memuat contoh rincian rekaman identitas medis pasien pada tabel **visit** sisi *node master* di dalam sistem rawat inap.


Server: localhost:3306 » Database: rawat_inap_vps » Table: visit

Browse | Structure | SQL | Search | Insert | Export | Import | Privileges | Operations | Tracking | Triggers

id	pasien_id	keluhan	pemeriksaan_fisik	diagnosa	tindakan	created_at	updated_at	uuid
75	67	demam, pilek dan badan terasa pegal pegal	NULL	influenza ringan	terapi simptomatik dan istirahat yang cukup	2026-06-15 09:11:18	2026-06-15 09:11:18	673fe931-57d7-4fb5-a5c9-e8243b44c80c
76	77	pegal diseluruh tubuh setelah aktivitas berat	NULL	Myalgia	pemberian analgesik dan istirahat	2026-06-15 09:11:59	2026-06-15 09:11:59	ba5c675b-4683-4579-90d4-522aab8f6e56
77	81	kontrol tekanan darah bulanan	NULL	Hipertensi ringan	evaluasi terapi dan edukasi untuk pola hidup sehat	2026-06-15 09:12:45	2026-06-15 09:12:45	2805ed7e-c7f6-4e76-904b-1fe914c885e9
78	87	badan lemas dan nafsu makan menurun	NULL	fatigue ringan	pemberian vitamin dan edukasi untuk menjaga pola hidup sehat	2026-06-15 09:13:26	2026-06-15 09:13:26	41b9e40f-b248-4d47-8e1f-e0228ba49141

Gambar 4.18 Potongan Baris Data pada Tabel visit *Node Slave*

Dilihat pada gambar 4.18 diatas menyajikan tampilan antarmuka pengelolaan basis data MySQL yang memuat contoh rincian rekaman identitas medis pasien pada tabel **visit** sisi *node slave* di dalam sistem rawat inap.

Dapat dilihat dari gambar 4.13 hingga Gambar 4.18, analisis integritas data menunjukkan bahwa relasi *foreign key* antar tabel tidak mengalami korupsi atau pergeseran indeks selama propagasi melalui jaringan *hybrid*. Keberhasilan ini mengonfirmasi bahwa skema replikasi yang diimplementasikan mampu menjamin sifat ACID (*Atomicity, Consistency, Isolation, and Durability*) basis data rumah

sakit. Dengan demikian, *node slave* memiliki salinan data yang sah, mutakhir, dan siap digunakan sebagai landasan proses pencadangan otomatis tingkat lanjut.

4.2.3 Analisis latensi replikasi

Untuk mendapatkan akurasi dalam satuan milidetik (ms), diterapkan instrumen *pt-heartbeat* pada *node Master* dengan interval 0,1 detik (100 ms) untuk menggantikan parameter bawaan MySQL (*Seconds_Behind_Master*) yang terbatas pada satuan detik. Di sisi *node slave*, skrip otomatis mengeksekusi kalkulasi selisih waktu berdasarkan *timestamp* biner murni dan mengekspor hasilnya ke format CSV (*Comma Separated Values*). Selanjutnya data tersebut diolah menggunakan statistik deskriptif berupa rumus *average time* (rata-rata) untuk menghitung rata-rata latensi replikasi secara keseluruhan. Pengujian dilakukan melalui dua skenario komprehensif, yaitu kondisi normal (*idle*) dan kondisi beban tinggi (saat proses *full backup*).

a. Skenario pengujian kondisi normal (*idle*)

Pada pengujian tahap pertama, dilakukan perekaman latensi selama periode 3 (tiga) menit berturut-turut pada saat antrean transaksi (*traffic*) jaringan *hybrid* berada dalam kondisi lengang atau tidak ada aktivitas input data rekam medis yang masif. Hasil ekstraksi dan agregasi statistik deskriptif dari ratusan baris data *log* tersebut disajikan pada tabel 4.5 berikut.

Tabel 4.5 Hasil Pengukuran Latensi Replikasi pada Kondisi Normal (*Idle*)

Parameter Statistik	Formula Excel	Nilai Hasil Pengujian	Indikasi Sistem
Latensi Minimum (*Min*)	=MIN(B2:B181)	4.3 ms	Kinerja puncak pipa replikasi pada kondisi ideal
Latensi Maksimum (*Max*)	=MAX(B2:B181)	105.3 ms	Jeda tunggu maksimal pemrosesan thread I/O
Rata-rata Latensi (*Average*)	=AVERAGE(B2:B181)	54 ms	Stabilitas rata-rata sinkronisasi data

Melalui perekaman selama ±3 menit, diperoleh total 180 sampel pengujian ($n = 180$) dengan jumlah nilai latensi sebesar 9.728,86 ms. Nilai pemusatan data dihitung menggunakan formula *Average Time* (persamaan 2.1) pada tinjauan teoritis, dengan penjabaran matematis sebagai berikut.

$$Average\ Time = \frac{9.728{,}86}{180} = 54\ ms$$

Dengan pendekatan kalkulasi yang sama terhadap seluruh populasi 180 sampel, didapatkan nilai rata-rata latensi representatif sebesar 54 ms, jauh di bawah batas toleransi 1 detik (1,000 ms). Berdasarkan tabel 4.5, arsitektur replikasi pada kondisi normal menunjukkan performa yang sangat presisi. Fluktuasi yang bergerak antara 4.3 ms hingga 105.3 ms, merupakan karakteristik wajar dari waktu tunggu (*wait time*) proses *Input/Output (I/O)* pada lingkungan VPS.

b. Skenario pengujian kondisi beban tinggi (saat proses *full backup*)

Pengujian tahap kedua dilakukan dengan merekam latensi saat beban kerja tinggi, yaitu melalui eksekusi skrip mariadb-*backup* di latar belakang *node slave*. Pengujian ini krusial untuk membuktikan apakah proses *hot backup* tersebut memicu pelambatan antrean replikasi secara signifikan. Hasil perhitungan statistik dari data perekaman pada skenario beban tinggi disajikan pada tabel 4.6 berikut.

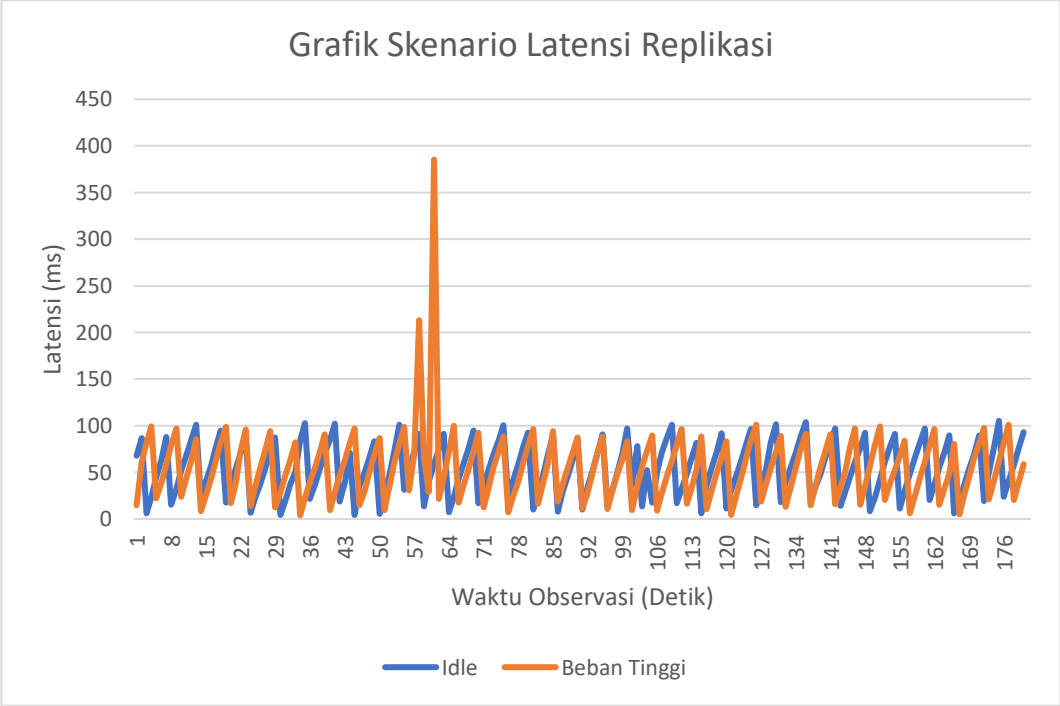
Tabel 4.6 Hasil Pengukuran Latensi Replikasi Saat Proses *Full Backup*

Parameter Statistik	**Formula Excel**	**Nilai Hasil Pengujian**	**Indikasi Sistem**
Latensi Minimum (*Min*)	=MIN(B2:B181)	3.9 ms	Kinerja puncak pipa replikasi pada kondisi ideal
Latensi Maksimum (*Max*)	=MAX(B2:B181)	385.3 ms	Jeda tunggu maksimal pemrosesan *thread I/O*
Rata-rata Latensi (*Average*)	=AVERAGE(B2:B181)	55.5 ms	Stabilitas rata-rata sinkronisasi data

Perekaman selama kurang lebih 3 menit menghasilkan total 180 sampel dan seluruh sampel dijumlahkan dan diperoleh angka 9.991,49 yang nilai pemusatan datanya dihitung menggunakan formula *average time* (persamaan 2.1) pada tinjauan teoritis, dengan penjabaran matematis sebagai berikut.

$$Average\ Time = \frac{9.991{,}49}{180} = 55.5\ ms$$

Meskipun sistem beroperasi di bawah tekanan konkurensi (beban CPU dan *Disk I/O* yang meningkat akibat proses penyalinan data *backup* fisik), parameter rata-rata latensi pada tabel 4.6 menunjukkan angka sebesar 55.5 ms. Terdapat peningkatan latensi maksimum yang mencapai 385.3 ms akibat *resource contention*, namun nilai *Average Time* tetap terjaga pada rentang operasional yang aman.



Gambar 4.19 Grafik Komparasi Fluktuasi Latensi

Data gambar 4.19 yang bersumber dari pencacatan instrumen *pt-heartbeat* dengan format CSV selama 3 menit observasi, menunjukkan kenaikan rata-rata latensi yang sangat minim dari kondisi normal (54 ms) ke skenario beban tinggi saat *full backup* (55.5 ms). Lonjakan maksimum sesaat hingga 385.3 ms mengindikasikan adanya kontensi sumber daya *Disk I/O* di *node slave*, yang

memicu antrean singkat pada *SQL Thread*. Namun, grafik yang segera kembali stabil membuktikan keandalan *catch-up mechanism* dalam mengisolasi beban pencadangan di server cadangan, sehingga operasional input data pada *node master* tetap berjalan *real-time* tanpa risiko penguncian tabel (*table locking*).

4.2.4 Analisis efisiensi durasi otomatisasi *backup*

Analisis durasi otomatisasi *backup* bertujuan mengevaluasi kinerja operasional mariadb-*backup* dan *cron job* pada Linux Ubuntu Server. Dalam ekosistem SIMRS, jendela eksekusi *backup* (*backup execution window*) merupakan metrik vital karena durasi yang terlalu lama dapat memicu degradasi performa sistem akibat lonjakan beban kerja *Disk Read/Write I/O* serta konsumsi memori server secara berkelanjutan.

Sesuai dengan konfigurasi *crontab* Linux yang telah direalisasikan pada tabel 4.3, memisahkan analisis durasi ini menjadi 2 (dua) kategori pencadangan, yaitu *full backup* harian yang dieksekusi setiap pukul 02:00 WIB dan *incremental backup* berkala yang dieksekusi pada pukul 09:00, 15:00, dan 21:00 WIB.

a. Analisis skenario *full backup* (pukul 02:00 WIB)

Proses *full backup* melibatkan penyalinan seluruh fisik berkas data basis data secara utuh. Pengujian dilakukan dengan mengamati catatan waktu (*timestamp*) mulai dan selesai eksekusi pada berkas *log* mariadb-*backup* selama beberapa sesi pengujian. Hasil pencatatan durasi waktu untuk skenario *full backup* dirangkum dalam tabel 4.7 berikut.

Tabel 4.7 Hasil Pengukuran Durasi Otomatisasi *Full Backup*

Sesi Pengujian (Backup ke-*i*)	**Waktu Mulai (WIB)**	**Waktu Selesai (WIB)**	**Durasi Penyelesaian (Detik)**	**Status Sistem**
16 Juni 2026	02:00:00	02:00:10	10	Sukses
17 Juni 2026	02:00:01	02:00:09	8	Sukses
18 Juni 2026	02:00:01	02:00:09	8	Sukses
19 Juni 2026	02:00:00	02:00:09	9	Sukses
20 Juni 2026	02:00:00	02:00:10	10	Sukses

Lanjutan Tabel 4.7

21 Juni 2026	02:00:01	02:00:10	9	Sukses
22 Juni 2026	02:00:00	02:00:10	10	Sukses

Berdasarkan penerapan rumus *average time* terhadap data populasi *full backup* (n = 7) pada tabel 4.7, maka perhitungan kuantitatifnya dihitung menggunakan formula *average time* (persamaan 2.1) pada tinjauan teoritis, dengan penjabaran matematis sebagai berikut.

$$Average\ Time = \frac{10 + 8 + 8 + 9 + 10 + 9 + 10}{7}$$

$$Average\ Time = \frac{64}{7} = 9{,}1\ detik$$

Dari hasil perhitungan tersebut, diperoleh rata-rata durasi penyelesaian untuk satu siklus *full backup* data rekam medis pasien adalah selama 9,1 detik (atau 9100 ms). Kecepatan eksekusi ini dinilai sangat efisien dan aman bagi lingkungan produksi rumah sakit karena waktu operasional server yang singkat meminimalkan beban kerja di pusat data.

b. Analisis skenario *incremental backup* (pukul 09:00, 15:00, dan 21:00 WIB)

Berbeda dengan *full backup*, metode *incremental backup* hanya menyalin berkas delta atau perubahan data klinis yang terjadi sejak pencadangan terakhir dilakukan. Pengujian ini krusial untuk membuktikan efisiensi ruang penyimpanan dan kecepatan waktu proses di tengah jam sibuk pelayanan rumah sakit. Hasil pencatatan durasinya dipaparkan pada tabel 4.8 berikut.

Tabel 4.8 Hasil Pengukuran Durasi Otomatisasi *Incremental Backup*

Sesi Pengujian (Backup ke-i)	**Jadwal Eksekusi**	**Waktu Mulai (WIB)**	**Waktu Selesai (WIB)**	**Durasi Penyelesaian (Detik)**
16 Juni 2026	Siklus Pagi	09:00:01	09:00:10	9
	Siklus Sore	15:00:01	15:00:09	8
	Siklus Malam	21:00:01	21:00:09	8

Lanjutan Tabel 4.8

17 Juni 2026	Siklus Pagi	09:00:00	09:00:09	9
	Siklus Sore	15:00:01	15:00:09	8
	Siklus Malam	21:00:00	21:00:08	8
18 Juni 2026	Siklus Pagi	09:00:01	09:00:14	13
	Siklus Sore	15:00:01	15:00:09	8
	Siklus Malam	21:00:01	21:00:09	8

Melalui kalkulasi statistik menggunakan formula *average time* (persamaan 2.1) pada tinjauan teoritis terhadap sampel *incremental backup* (n = 9), diperoleh perhitungan sebagai berikut.

$$Average\ Time = \frac{9 + 8 + 8 + 9 + 8 + 8 + 13 + 8 + 8}{9}$$

$$Average\ Time = \frac{79}{9} = 8{,}7\ detik$$

Berdasarkan komparasi antara durasi eksekusi *full backup* rata-rata (9,1 detik) dengan *incremental backup* rata-rata (8,7 detik), tingkat efisiensi waktu pemrosesan yang dicapai sistem dapat diukur. Tingkat efisiensi waktu (*time efficiency rate*) ini dihitung menggunakan formula persentase perbandingan selisih waktu operasional terhadap waktu dasar (persamaan 2.2) pada tinjauan teoritis dan diperoleh perhitungan sebagai berikut.

$$Tingkat\ Efisiensi = \frac{T_{full} - T_{incremental}}{T_{full}}\ \text{X}\ 100\%$$

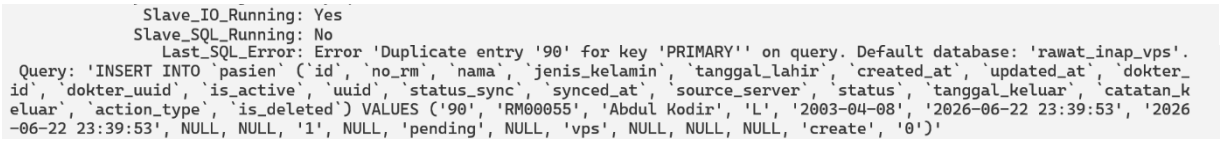
$$Tingkat\ Efisiensi = \frac{9{,}1 - 8{,}7}{9{,}1}\ \text{X}\ 100\% = 4{,}39\%$$

Hasil perhitungan kuantitatif di atas menunjukkan bahwa penerapan algoritma *incremental backup* pada jam sibuk pelayanan rumah sakit (pukul 09:00, 15:00, dan 21:00 WIB) berhasil memberikan tingkat efisiensi waktu eksekusi sebesar 4,39% per siklus pencadangan jika dibandingkan dengan eksekusi penyalinan fisik secara penuh. Meskipun persentase pemangkasan waktu ini berada pada skala hitungan detik, dalam ekosistem produksi rumah sakit dengan laju

pertumbuhan akumulasi data medis yang masif dan eksponensial di masa depan, rasio efisiensi ini akan sangat esensial dalam mereduksi utilisasi beban baca-tulis perangkat keras (*Disk I/O*) peladen sekunder.

4.2.5 Pengujian penanganan kegagalan replikasi

Simulasi anomali kritis berupa berhentinya sinkronisasi akibat konflik integritas data dilakukan dengan mengondisikan pelanggaran hak akses *read-only* pada *node slave* secara sengaja, dengan melakukan injeksi data pemeriksaan pasien ilegal (misalnya ID: 90, No. RM: RM00055) secara langsung ke dalam basis data *node slave*. Setelah itu, simulasi dilanjutkan dengan menginput data pasien sah menggunakan *Primary Key* yang sama (ID: 90) melalui aplikasi SIMRS terminal rawat inap yang terhubung ke *node master*. Injeksi paralel ini secara logis memicu benturan identitas (*duplicate key*) ketika *relay log* pada *slave* mencoba menerapkan transaksi dari *master*, yang menyebabkan proses replikasi terhenti meski kedua server berstatus *running*. Hasil pemantauan parameter saat konflik terjadi didokumentasikan pada gambar 4.20 dan tabel 4.9 berikut.



```
             Slave_IO_Running: Yes
            Slave_SQL_Running: No
               Last_SQL_Error: Error 'Duplicate entry '90' for key 'PRIMARY'' on query. Default database: 'rawat_inap_vps'.
 Query: 'INSERT INTO `pasien` (`id`, `no_rm`, `nama`, `jenis_kelamin`, `tanggal_lahir`, `created_at`, `updated_at`, `dokter_
id`, `dokter_uuid`, `is_active`, `uuid`, `status_sync`, `synced_at`, `source_server`, `status`, `tanggal_keluar`, `catatan_k
eluar`, `action_type`, `is_deleted`) VALUES ('90', 'RM00055', 'Abdul Kodir', 'L', '2003-04-08', '2026-06-22 23:39:53', '2026
-06-22 23:39:53', NULL, NULL, '1', NULL, 'pending', NULL, 'vps', NULL, NULL, NULL, 'create', '0')'
```

Gambar 4.20 *Log* Eror *Duplicate Entry* pada Status Replikasi *Node Slave*

Berdasarkan *log* eror *Duplicate Entry* yang terekam pada gambar 4.20 di atas, observasi lebih lanjut dilakukan untuk menganalisis dampak anomali tersebut terhadap status *thread* replikasi, yang tahapan evaluasinya diuraikan secara rinci pada tabel 4.9 berikut.

Tabel 4.9 Evaluasi Skenario Kegagalan Sinkronisasi (Konflik Data)

Tahap Observasi	**Tindakan / Kondisi Sistem**	**Status *Slave_IO_Running***	**Status *Slave_SQL_Running***	**Indikasi Sistem & Output Observasi**
1	Injeksi data ilegal ke *node slave*	**Yes**	**Yes**	Replikasi masih berjalan normal, menunggu *log* baru dari *Master*.

Lanjutan Tabel 4.9

2	Injeksi data sah ke *node master* dengan ID yang sama	**Yes**	**No**	*Thread* SQL terhenti karena benturan data. Memunculkan peringatan: **Error 1062: Duplicate entry '90' for key 'PRIMARY'.**
3	Pemulihan (*Disaster Recovery*) oleh *Administrator*	**Yes**	**Yes**	Data ilegal di *Slave* dihapus secara manual. Sistem melakukan *catch-up* sinkronisasi untuk data yang sempat tertahan.

Hasil observasi tabel 4.9 menunjukkan bahwa saat konflik *Primary Key* terjadi, parameter *Slave_IO_Running* tetap berstatus *Yes*, tetapi *Slave_SQL_Running* berubah menjadi *No* akibat *error 1062*. Temuan ini mengungkap celah arsitektural krusial, yakni kegagalan replikasi yang tidak ditangani akan membuat rutinitas *cron job* menghasilkan berkas cadangan berisi data usang (*stale data*) karena *log* terbaru tertahan di *relay log*. Untuk memulihkan sistem tanpa kehilangan antrean data pasien, dilakukan intervensi server cadangan melalui prosedur *disaster recovery* dengan menghentikan replikasi sementara (**STOP SLAVE;**), menghapus baris data penyebab konflik (**DELETE FROM pasien WHERE id = 90;**), dan mengaktifkan kembali mesin sinkronisasi (**START SLAVE;**).

4.2.6 Pengujian ketersediaan (*failover*)

Pengujian ketersediaan (*failover*) dilakukan secara eksperimental untuk memverifikasi ketangguhan sistem dalam menjaga layanan rekam medis saat terjadi gangguan pada server utama. Simulasi kegagalan terkontrol dilakukan dengan mematikan layanan MySQL pada *node master* secara paksa menggunakan perintah **systemctl stop mariadb.** Segera setelah server utama padam, perilaku komponen replikasi dan fungsionalitas akses data pada aplikasi diuji untuk memastikan *node*

slave siap mengambil alih peran melayani kueri baca (*read-only*), yang hasilnya dirangkum pada tabel 4.10 berikut.

Tabel 4.10 Hasil Pengujian Skenario Ketersediaan (*Failover*) Sistem

Tahap	**Kondisi *Node Master***	**Aktivitas Akses Aplikasi**	**Parameter Status Replikasi *Node Slave***	**Hasil Observasi (*Output*)**	**Status**
1	*Active* (*Running*)	Kueri Tulis (*Write*) & Baca (*Read*)	*Slave_IO_Running: Yes* *Slave_SQL_Running: Yes*	Data tersimpan pada *master* dan tersinkron-isasi sempurna ke *slave*.	Sukses
2	*Down* (*Stopped*)	Kueri Tulis (*Write*) Baru	*Slave_IO_Running: Connecting* *Slave_SQL_Running: Yes*	Kueri gagal dieksekusi; koneksi ke server *master* terputus (*Connection Refused*).	Terkondisi
3	*Down* (*Stopped*)	Kueri Baca (*Read*) Data Lama	*Slave_IO_Running: Connecting* *Slave_SQL_Running: Yes*	Aplikasi berhasil menampil-kan rekam medis pasien mengguna-kan data dari *node slave*.	Sukses

Sebagai bukti validasi dari ringkasan observasi pada tabel 4.10 di atas, kondisi aktual parameter *thread* replikasi pada antarmuka baris perintah (CLI) saatpeladen utama (*node master*) berstatus aktif disajikan pada gambar 4.21 dan gambar 4.22 berikut.

```
MariaDB [(none)]> SHOW SLAVE STATUS\G;
*************************** 1. row ***************************
               Slave_IO_State: Waiting for master to send event
                  Master_Host: [redacted]
                  Master_User: [redacted]
                  Master_Port: 3306
                Connect_Retry: 60
              Master_Log_File: mysql-bin.000007
          Read_Master_Log_Pos: 342
               Relay_Log_File: mysqld-relay-bin.000041
                Relay_Log_Pos: 641
        Relay_Master_Log_File: mysql-bin.000007
             Slave_IO_Running: Yes
            Slave_SQL_Running: Yes
              Replicate_Do_DB:
```

Gambar 4.21 Status *Slave* Ketika *Node Master* Aktif

Sebaliknya, ketika layanan basis data pada peladen utama (*node master*) dimatikan secara paksa untuk menyimulasikan skenario kegagalan, perubahan respons dan status *thread* replikasi pada *node slave* dapat diamati secara langsung pada gambar 4.22 berikut.

```
MariaDB [(none)]> SHOW SLAVE STATUS\G;
*************************** 1. row ***************************
               Slave_IO_State: Reconnecting after a failed master event read
                  Master_Host: [redacted]
                  Master_User: [redacted]
                  Master_Port: 3306
                Connect_Retry: 60
              Master_Log_File: mysql-bin.000006
          Read_Master_Log_Pos: 312412
               Relay_Log_File: mysqld-relay-bin.000039
                Relay_Log_Pos: 312711
        Relay_Master_Log_File: mysql-bin.000006
             Slave_IO_Running: Connecting
            Slave_SQL_Running: Yes
              Replicate_Do_DB:
```

Gambar 4.22 Status *Slave* Ketika *Node Master Down*

Berdasarkan data observasi pada gambar 4.21, gambar 4.22, dan tabel 4.10, saat *node master* lumpuh, transaksi penulisan data baru dihentikan sementara, namun pemanggilan kembali rekam medis historis (*read-only retrieval*) tetap berjalan normal karena sistem otomatis mengalihkan *pointer* koneksi baca ke alamat IP *node slave*. Pengalihan beban ini berhasil menjamin operasional diagnosis tenaga medis menggunakan replika data terakhir yang sinkron. Keberhasilan pengujian *failover* ini membuktikan secara pengamatan bahwa topologi basis data terdistribusi pada jaringan *hybrid* mampu meminimalkan risiko *single point of failure* sekaligus bisa menjadi solusi *disaster recovery* yang andal.

4.3 Pembahasan

Bagian pembahasan ini menjelaskan hasil pengujian dan analisis performa sistem replikasi basis data dan otomatisasi pencadangan yang telah diimplementasikan pada jaringan *hybrid*. Evaluasi difokuskan pada tingkat konsistensi data, stabilitas latensi sinkronisasi, efisiensi waktu eksekusi *backup*, serta resiliensi sistem dalam skenario kegagalan (*failover*).

4.3.1 Analisis keandalan dan konsistensi replikasi *master-slave*

Penerapan topologi *master-slave* pada jaringan *hybrid* rumah sakit berhasil mendistribusikan data dengan tingkat yang konsisten dan nilai selisih (*delta*) nol berdasarkan uji injeksi 50 *records* rekam medis. Keberhasilan transmisi tanpa korupsi *foreign key* atau pergeseran indeks ini membuktikan bahwa arsitektur sistem mampu menjamin sifat ACID (*Atomicity, Consistency, Isolation, and Durability*). Kinerja tersebut menegaskan bahwa penggunaan MariaDB pada lingkungan VPS efektif menghilangkan risiko *single point of failure*, sekaligus memenuhi tuntutan ketersediaan data berkelanjutan dalam modernisasi SIMRS.

4.3.2 Latensi jaringan terhadap proses *hot hackup*

Pengukuran latensi menggunakan instrumen *pt-heartbeat* membuktikan kualitas dan resiliensi jaringan *hybrid* dalam menangani lalu lintas data medis. Pada kondisi normal (*idle*), rata-rata latensi tercatat sebesar 54 ms dengan nilai maksimum 105.3 ms. Saat skenario beban tinggi akibat eksekusi *full backup* di *node slave*, terjadi kontensi sumber daya *Disk I/O* dan CPU yang memicu lonjakan latensi maksimum secara tajam hingga 385.3 ms. Namun, analisis menggunakan *average time* menunjukkan bahwa akumulasi rata-rata latensi hanya bergeser sedikit ke angka 55.5 ms (selisih kenaikan hanya 1.5 ms).

Stabilitas rata-rata latensi yang berada jauh di bawah ambang batas kritis (1,000 ms) ini mengonfirmasi dua keberhasilan teknis secara bersamaan, efektivitas *catch-up mechanism* yang mempercepat pemulihan antrean replikasi tanpa memicu kegagalan pipa komunikasi, serta keberhasilan operasi *non-locking* dari mariadb-*backup* yang mencegah efek penguncian tabel (*table locking*). Hasil ini membuktikan bahwa infrastruktur *master-slave* Penulis mampu menjamin

operasional input data klinis secara *real-time* tanpa disrupsi (*zero-downtime*), terlepas dari adanya aktivitas pencadangan berat di latar belakang.

4.3.3 Efisiensi otomatisasi *backup* menggunakan *average time*

Otomatisasi menggunakan utilitas *cron job* berhasil menggantikan keharusan intervensi manual administrator dalam mengamankan data medis. Pengukuran melalui metode *average time* membuktikan rata-rata durasi *full backup* harian sebesar 9,1 detik dan *incremental backup* berkala sebesar 8,7 detik. Berdasarkan komparasi tersebut, sistem arsitektur yang dibangun terbukti menghasilkan tingkat efisiensi waktu komputasi sebesar 4,39% pada jam-jam operasional puncak layanan rawat inap. Durasi eksekusi yang konsisten berada di bawah ambang batas kritis 10 detik ini merepresentasikan rasio efisiensi yang tinggi, sehingga secara efektif mampu mencegah degradasi performa (*bottleneck*) akibat utilisasi *Disk Read/Write I/O* yang berlebihan pada peladen cadangan.

4.3.4 Implikasi kegagalan sinkronisasi replikasi

Pengujian *fault injection* mengungkap celah krusial berupa anomali redudansi data kedaluwarsa, konflik integritas data (*error 1062*) menyebabkan proses aplikatif kueri (*Slave_SQL_Running*) lumpuh total meskipun pipa komunikasi data tetap aktif. Kondisi ini menahan data rekam medis baru di dalam *relay log* tanpa diterapkan ke tabel produksi cadangan. Karena utilitas *cron job* bekerja murni berdasarkan penjadwalan waktu tanpa pemantauan kesehatan internal basis data, skrip mariadb-*backup* akan tetap dieksekusi otomatis pada pukul 02:00 WIB dan menghasilkan status *Success*. Hal ini menciptakan risiko tinggi berupa ilusi keamanan (*false sense of security*), sebab berkas cadangan yang terbentuk sebenarnya berisi data usang (*stale data*) yang telah mengalami desinkronisasi dari server utama.

Temuan ini membuktikan bahwa otomatisasi *backup* memerlukan mekanisme pengawasan status replikasi (*replication state monitoring*) dan sistem peringatan dini *real-time* guna menjamin validitas serta kemutakhiran data rekam medis pasien saat terjadi bencana (*disaster recovery*).

4.3.5 Ketersediaan data tinggi (*high availability*) dan *failover*

Uji simulasi kegagalan (*failover test*) menjadi alasan akhir keandalan arsitektur ini bisa sebagai solusi pemulihan bencana (*disaster recovery*). Saat layanan basis data utama lumpuh (*down*), kueri penulisan data baru tertahan demi mencegah inkonsistensi, namun aplikasi rawat inap berhasil dialihkan ke *node slave* untuk mempertahankan fungsionalitas pemanggilan data (*read-only retrieval*). Akses riwayat medis pasien tetap bisa berjalan hampir tanpa adanya jeda (*zero-downtime*), membuktikan bahwa arsitektur jaringan *hybrid* ini berhasil mewujudkan tingkat ketersediaan data tinggi (*high availability*) yang ditargetkan dalam penelitian.